3. Запускаем скрипты

Тебе понадобятся три окна терминала (PowerShell или cmd) на твоем компе. В каждом подключись к малинке по новому Wi-Fi адресу: `ssh pi@НОВЫЙ_IP_РОБОТА`.

- **Окно 1 (Поднимаем ядро):**

Bash

```bash
docker start xiao_ros_brain
docker exec -it xiao_ros_brain bash -c "source /opt/ros/noetic/setup.bash &&
```

- **Окно 2 (Сервер Unity):**

Bash

```bash
:os_tcp_endpoint default_server_endpoint.py --tcp_ip 0.0.0.0 --tcp_port 10000"
```

- **Окно 3 (Твой боевой скрипт):**

Bash

```bash
docker exec -it xiao_ros_brain bash -c "source /opt/ros/noetic/setup.bash &&
```

айпишник 192.168.1.1
порт 10000
docker start xiao_ros_brain
docker exec -it xiao_ros_brain bash -c "source /opt/ros/noetic/setup.bash && roscore"

docker exec -it xiao_ros_brain bash -c "source /opt/ros/noetic/setup.bash && source /root/catkin_ws/devel/setup.bash && rosrun ros_tcp_endpoint default_server_endpoint.py --tcp_ip 0.0.0.0 --tcp_port 10000"

docker exec -it xiao_ros_brain bash -c "source /opt/ros/noetic/setup.bash && python3 /root/unity.py"

Техническая документация: Интеграция ROS (Raspberry Pi) и Unity Robotics Hub

Данный документ описывает полный цикл настройки программно-аппаратного комплекса для управления роботом через Unity, используя архитектуру клиент-сервер на базе протокола TCP/IP.

1. Первичное подключение к Raspberry Pi

Плата поставляется без графического интерфейса (headless) и управляется удаленно через протокол SSH.

1.1. Определение IP-адреса платы

- **Режим Ethernet (по кабелю):** Подключите кабель от роутера. IP-адрес выдается автоматически (DHCP). Узнать его можно в админ-панели роутера или командой `hostname -I` (при наличии временного монитора).
- **Режим Wi-Fi (точка доступа):** Включите робота. Он создаст сеть (например, `XiaoR_WIFI`). Подключитесь к ней с ПК. Откройте командную строку Windows (`cmd`), выполните `ipconfig` и найдите **Основной шлюз** (Default Gateway) беспроводного адаптера. Как правило, это `192.168.1.1`.

1.2. Подключение

В командной строке выполните:

```
ssh pi@192.168.1.1
```

(Пароль по умолчанию: `raspberry`. При вводе символы не отображаются на экране).

1.3. Настройка интернета (для скачивания пакетов)

Если плата подключена по Wi-Fi к компьютеру, у нее нет выхода в глобальный интернет для скачивания пакетов. Для первичной настройки необходимо временно подключить плату кабелем Ethernet к роутеру с интернетом.

2. Установка зависимостей: Python, Docker и ROS

Устанавливать ROS непосредственно на операционную систему Raspberry Pi не рекомендуется из-за конфликтов зависимостей. Мы используем контейнеризацию.

Восстановление доступа к репозиториям Для старых версий Raspberry Pi OS необходимо переключить менеджер пакетов `apt` на архивные зеркала, иначе скачивание пакетов будет заблокировано. Выполните команду:

```
echo "deb http://legacy.raspbian.org/raspbian/ buster main contrib
non-free rpi" | sudo tee /etc/apt/sources.list
```

2.1. Установка Python

Python 3 обычно предустановлен в Raspberry Pi OS. Для обновления/проверки выполните:

```
sudo apt update
sudo apt install python3 python3-pip
```

2.2. Установка Docker

Загрузите и выполните официальный установочный скрипт:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

Добавьте пользователя в группу docker, чтобы не вводить `sudo` каждый раз:

```
sudo usermod -aG docker pi
```

(После выполнения команды требуется перезагрузка: `sudo reboot`).

2.3. Создание Docker-контейнера `xiao_ros_brain`

Архитектурная справка: Несмотря на то, что аппаратный процессор Raspberry Pi 4B поддерживает 64-битные инструкции (ARM64), установленная операционная система робота является 32-битной (ARMv7 / `armhf`). Это накладывает жесткое ограничение на весь стек верхнего уровня: установка ROS 2 Humble невозможна из-за несовместимости манифестов. В качестве решения используется ROS 1 Noetic, который нативно поддерживает 32-битные системы и полностью совместим с пакетами Unity.

Запуск контейнера ROS 1 Noetic Команда загружает 32-битный образ ROS и предоставляет контейнеру права суперпользователя для доступа к шине I2C::

```
docker run -it --net=host --privileged --name xiao_ros_brain
arm32v7/ros:noetic-ros-base bash
```

(Примечание: `xiaorgeek/ros-noetic` — это пример названия образа, содержащего ROS Noetic и библиотеку драйверов `xr_motor`. Замените на точное имя вашего заводского образа).

3. Сборка рабочего пространства (Workspace) внутри Docker

Все последующие команды данного раздела выполняются **внутри** запущенного контейнера (`root@raspberrypi:/#`).

Установка системных зависимостей

```
apt-get update && apt-get install -y git python3-pip
```

Инициализация и сборка Catkin Workspace Создание структуры директорий, скачивание официального TCP-коннектора от Unity и его компиляция:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
git clone https://github.com/Unity-Technologies/ROS-TCP-Endpoint.git
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

Фоновый запуск ядра ROS Главный координатор системы (`roscore`) должен быть запущен для работы любых узлов. Запускаем его в фоновом режиме (с амперсандом), чтобы освободить терминал:

```
roscore &
```

(Нажмите Enter после завершения вывода логов, чтобы вернуть строку ввода).

Запуск существующего контейнера

После перезагрузки платы контейнер необходимо запускать вручную:

```
docker start xiao_ros_brain
```

4. Настройка программного моста (Скрипт моторов)

4.1. Создание скрипта `unity.py`

Создайте файл в домашней директории платы:

```
nano unity.py
```

Вставьте код для конвертации команд из топика `/cmd_vel` в аппаратные сигналы:

```
#!/usr/bin/env python3
import sys
import rospy
from geometry_msgs.msg import Twist

sys.path.append('/root/XiaoRGeek')
from xr_motor import RobotDirection

robot = RobotDirection()

def cb(msg):
    linear = msg.linear.x
    angular = msg.angular.z

    speed = int(abs(linear) * 100)
    turn = int(abs(angular) * 50)
    left_speed = speed
    right_speed = speed

    if linear > 0.1:
        robot.m1m2_forward()
        robot.m3m4_forward()
        if angular > 0.1:
            left_speed = max(0, speed - turn)
            right_speed = min(100, speed + turn)
        elif angular < -0.1:
            left_speed = min(100, speed + turn)
            right_speed = max(0, speed - turn)
    elif linear < -0.1:
        robot.m1m2_reverse()
        robot.m3m4_reverse()
    elif abs(angular) > 0.1:
        left_speed = turn
        right_speed = turn
        if angular > 0:
            robot.m1m2_reverse()
            robot.m3m4_forward()
        else:
            robot.m1m2_forward()
            robot.m3m4_reverse()
    else:
        robot.m1m2_stop()
```

```
        robot.m3m4_stop()
        left_speed = 0
        right_speed = 0

    robot.set_speed(1, max(0, min(100, left_speed)))
    robot.set_speed(2, max(0, min(100, right_speed)))

def listener():
    rospy.init_node('unity_bridge', anonymous=True)
    rospy.Subscriber("/cmd_vel", Twist, cb)
    print("Готов к приему команд")
    rospy.spin()

if __name__ == '__main__':
    listener()
```

Сохраните файл (`Ctrl+O`, `Enter`, `Ctrl+X`).

4.2. Копирование и исправление ошибок в Docker

Скопируйте скрипт внутрь контейнера:

```
docker cp unity.py xiao_ros_brain:/root/unity.py
```

Чтобы избежать ошибки `Address already in use` (занятый порт Bluetooth), смените порт в заводской библиотеке с 2002 на 2005:

```
docker exec -it xiao_ros_brain sed -i 's/2002/2005/g'
/root/XiaoRGeek/xr_config.py
```

5. Настройка клиента в Unity (Устранение DDoS)

Импорт и генерация ROS-сообщений Для того чтобы Unity понимала формат сообщений `geometry_msgs/Twist`, необходимо скомпилировать их C#-эквиваленты:

1. Скачайте ZIP-архив репозитория `common_msgs` (ветка Noetic) и распакуйте его на ПК.
2. В Unity откройте **ROS Message Browser**.
3. В поле *ROS message path* укажите путь к распакованной папке `common_msgs-noetic-devel`.
4. Раскройте директорию `geometry_msgs -> msg`, найдите `Twist.msg` и нажмите **Build msg**.

Чтобы Unity не переполняла буфер Raspberry Pi (ошибка `Queue full`), необходимо ограничить частоту отправки команд до 10 Гц.

1. Укажите IP-адрес `192.168.1.1` в компоненте `ROS_Manager`.
2. Повесьте скрипт `RobotController.cs` на **один** пустой объект сцены.

Код C#:

```
using UnityEngine;
using Unity.Robotics.ROSTCPConnector;
using RosMessageTypes.Geometry;

public class RobotController : MonoBehaviour
{
    ROSConnection ros;
    public string topicName = "/cmd_vel";
    public float linearSpeed = 0.5f;
    public float angularSpeed = 1.0f;
    private float timeElapsed = 0f;
    public float publishMessageFrequency = 0.1f;

    void Start()
    {
        ros = ROSConnection.GetOrCreateInstance();
        ros.RegisterPublisher<TwistMsg>(topicName);
    }

    void Update()
    {
        timeElapsed += Time.deltaTime;
        if (timeElapsed > publishMessageFrequency)
        {
            float moveForward = Input.GetAxis("Vertical") * linearSpeed;
            float turnRight = Input.GetAxis("Horizontal") *
angularSpeed;

            TwistMsg twist = new TwistMsg();
            twist.linear.x = moveForward;
            twist.angular.z = -turnRight;

            ros.Publish(topicName, twist);
            timeElapsed = 0f;
        }
    }
```

```
}
```

6. Инструкция по запуску работы комплекса

Для инициализации обмена данными и управления необходимо выполнить команды внутри ROS-окружения (`/opt/ros/noetic/setup.bash`). Откройте три окна терминала и выполните:

Терминал 1: Ядро ROS

```
docker start xiao_ros_brain
docker exec -it xiao_ros_brain bash -c "source
/opt/ros/noetic/setup.bash && roscore"
```

Терминал 2: TCP Endpoint сервер связи

```
docker exec -it xiao_ros_brain bash -c "source
/opt/ros/noetic/setup.bash && source /root/catkin_ws/devel/setup.bash &&
rosrun ros_tcp_endpoint default_server_endpoint.py --tcp_ip 0.0.0.0
--tcp_port 10000"
```

Терминал 3: Управление моторами

```
docker exec -it xiao_ros_brain bash -c "source
/opt/ros/noetic/setup.bash && python3 /root/unity.py"
```

После появления в Терминале 3 надписи `"Готов к приему команд"`, можно запускать сцену в Unity.